

KathaQuest: One-Day Hackathon Build Plan

Instructions for Codex

Act as the lead product engineer for this hackathon project.

Build a working, polished MVP that qualifies for both:

1. VideoDB: Unlock the Footage Hackathon
2. Agents of SigNoz Hackathon

Work autonomously. Do not stop for minor clarifications. Make sensible technical decisions, prioritise a working end-to-end demo, and reduce scope whenever necessary.

Important operating rules:

- First inspect the existing repository before changing anything.
- Preserve useful existing code.
- Use TypeScript with strict type checking.
- Keep architecture simple enough to finish within the available time.
- Build one complete vertical workflow before adding secondary features.
- Do not use mocked VideoDB results in the final demonstration.
- Use actual archived video or audio files.
- Keep all API credentials in environment variables.
- Never commit secrets.
- Create and maintain a `BUILD_STATUS.md` checklist.
- Commit after each major milestone.
- Reserve the final three hours for testing, documentation and submission.
- When a stretch feature threatens the core demo, remove the stretch feature.

1. Submission Clock

Assume less than 16 hours remain.

Hard priorities

1. Make VideoDB ingestion, indexing, search and playable clip generation work.
2. Instrument the complete workflow using OpenTelemetry and SigNoz.
3. Build a simple but polished user interface.
4. Demonstrate one real failure and automatic provider fallback.
5. Complete the public repository, README, demo recording and submission materials.

Do not spend time building unnecessary platform features.

2. Product Overview

Product name

KathaQuest

Tagline

Turn textbook chapters into multilingual video adventures using real educational footage.

One-line pitch

KathaQuest converts a textbook chapter into short video lessons for children by retrieving and combining relevant moments from real educational footage, while SigNoz observes every AI call, failure, latency spike and cost.

Problem

Books and textbooks can feel difficult or uninteresting to children. Existing AI video generators often produce unreliable or completely synthetic explanations.

At the same time, thousands of useful lectures, documentaries and educational recordings already exist, but their best moments are difficult to find.

Solution

KathaQuest uses a chapter as a learning roadmap.

It:

1. Reads the chapter.
 2. Extracts three important concepts.
 3. Searches a trusted educational video archive.
 4. Finds exact moments that explain each concept.
 5. Creates three short playable learning episodes.
 6. Explains them in English or Hindi.
 7. Allows the child to ask a voice question.
 8. Returns a simple answer with supporting video evidence.
 9. Tracks the complete AI pipeline inside SigNoz.
-

3. Hackathon Alignment

VideoDB requirements

The project must:

- Use actual archived video or audio.
- Produce structured, searchable or actionable media intelligence.
- Include a working demo.
- Include a public GitHub repository.
- Include a description of no more than 200 words explaining the product and VideoDB usage.
- Avoid synthetic or mocked source media in the final demonstration.

The hackathon judges technical execution at 40%, creativity at 30%, and depth of VideoDB usage at 30%. The event specifically lists searchable educational archives and personalised study clips as valid use cases.

SigNoz requirements

The project must:

- Use or integrate with SigNoz.
- Install SigNoz through Foundry.
- Include `casting.yaml`.
- Include `casting.yaml.lock`.
- Disclose the use of AI coding assistants.
- Use OpenTelemetry deeply where possible.
- Preferably demonstrate traces, metrics, logs, dashboards, alerts and SigNoz MCP.

The strongest submission track is:

Track 01: AI & Agent Observability

SigNoz judging includes impact, creativity, technical excellence, use of SigNoz, user experience and presentation quality.

4. Final MVP Scope

Must-have features

A. Chapter input

Allow the user to:

- Upload a text-based PDF.
- Or use a preloaded sample chapter.
- Select age group.

- Select English or Hindi.

For the final demo, preload one chapter about:

Volcanoes

B. Chapter understanding

Use an LLM to extract exactly three concepts:

1. What is a volcano?
2. Why do volcanoes erupt?
3. What happens during an eruption?

Return structured JSON containing:

- Concept title
- Child-friendly explanation
- Video search query
- Quiz question
- Correct answer
- Three incorrect options

C. Real educational video archive

Use 5–8 real videos about volcanoes.

The videos must be:

- Public-domain
- Creative Commons
- Owned by the team
- Or otherwise permitted for demonstration

Store the source title, URL and licence information in:

```
data/demo-videos.json
```

D. VideoDB pipeline

For every archive video:

1. Upload it to VideoDB.
2. Create spoken-word understanding.
3. Create visual or scene understanding.
4. Index the resulting transcript and scene data.
5. Search across the collection for each chapter concept.
6. Retrieve the best timestamped moments.
7. Compile the results into a playable HLS stream.

VideoDB supports spoken-word and visual indexes, semantic search, timestamped streams and timeline-based video composition.

E. Three learning episodes

Generate exactly three episode cards.

Each card must display:

- Episode title
- Short explanation
- VideoDB-generated stream
- Duration
- Source video name
- “Why this clip?” explanation
- Search relevance or confidence indicator

F. English and Hindi narration

Use:

- Sarvam AI as the primary Hindi voice provider.
- ElevenLabs as the primary English voice provider.
- Each provider as a fallback for the other where supported.

For the MVP, narration does not have to be permanently mixed into the video.

It may:

- Play alongside the episode.
- Play before the video.
- Or be controlled through a “Listen to explanation” button.

Sarvam provides text-to-speech, speech-to-text, translation and multilingual APIs. ElevenLabs provides text-to-speech with selectable voices and multilingual models.

G. Ask a question

Allow the child to ask a question using:

- Microphone input
- Or typed text

For Hindi voice input:

1. Record a short audio file in the browser.
2. Send it to Sarvam Speech-to-Text.
3. Extract the transcript.
4. Use the transcript to search VideoDB.
5. Return:

6. A simple text answer

7. The most relevant playable video moment

Sarvam's speech-to-text endpoint supports Indian languages, English, language detection and several transcription modes.

H. Quiz and revision

After watching the episodes:

1. Show three multiple-choice questions.
2. Record wrong answers.
3. Generate a "Revision Reel" from VideoDB clips related to the incorrectly answered concepts.
4. Play the revision reel as one compiled stream.

I. SigNoz observability

Create an end-to-end trace for every lesson-generation request.

The trace must make the entire agent workflow visible.

J. Failure demonstration

Add a clearly marked developer-only button:

Simulate ElevenLabs Failure

When activated:

1. ElevenLabs must throw a controlled error.
2. The error must appear in the SigNoz trace.
3. An error counter must increase.
4. The system must automatically use Sarvam TTS.
5. The lesson must still complete successfully.
6. The UI must display:
7. "Primary voice provider failed"
8. "Recovered using Sarvam AI"

This is important for the SigNoz demonstration.

5. Features Explicitly Out of Scope

Do not build these during the hackathon:

- User authentication

- Payments
- Parent accounts
- Teacher accounts
- Full book conversion
- More than one subject
- More than two languages
- Complex student analytics
- Social features
- Multiplayer learning
- AI avatars
- Fully synthetic video generation
- Mobile applications
- Large database infrastructure
- Advanced role-based permissions
- Production-grade content moderation
- A marketplace
- A full school management dashboard

6. Recommended Technology Stack

Layer	Technology
Web application	Next.js with App Router
Language	TypeScript
UI	Tailwind CSS and shadcn/ui
Video playback	HLS.js
Media intelligence	VideoDB Node.js SDK
Observability	SigNoz
Telemetry	OpenTelemetry
Chapter reasoning	OpenAI API or another available structured-output LLM
Hindi STT and TTS	Sarvam AI
Translation	Sarvam AI
English narration	ElevenLabs
PDF extraction	<code>unpdf</code> , <code>pdf-parse</code> or equivalent
Validation	Zod
Logging	Pino
Temporary persistence	Local JSON or SQLite
Deployment	Vercel for the web demo, local Docker for SigNoz
Source control	GitHub

Architecture decision

Use a **single Next.js repository**.

Do not build a separate Python or FastAPI backend unless existing code already depends on it.

Use Next.js route handlers for external API calls.

This saves time and keeps deployment simple.

7. Suggested Repository Structure

```
kathaquest/
├── app/
│   ├── page.tsx
│   ├── generate/page.tsx
│   ├── lesson/[lessonId]/page.tsx
│   ├── observability/page.tsx
│   └── api/
│       ├── lessons/generate/route.ts
│       ├── lessons/[lessonId]/route.ts
│       ├── videos/seed/route.ts
│       ├── questions/ask/route.ts
│       ├── narration/route.ts
│       ├── quiz/submit/route.ts
│       ├── demo/failure/route.ts
│       └── health/route.ts
├── components/
│   ├── chapter-upload.tsx
│   ├── generation-progress.tsx
│   ├── episode-card.tsx
│   ├── hls-player.tsx
│   ├── voice-question.tsx
│   ├── quiz.tsx
│   ├── provider-status.tsx
│   └── trace-link.tsx
├── lib/
│   ├── pipeline.ts
│   ├── videodb.ts
│   ├── chapter-parser.ts
│   ├── llm.ts
│   ├── sarvam.ts
│   ├── elevenlabs.ts
│   ├── narration-router.ts
│   ├── telemetry.ts
│   ├── logger.ts
│   └── storage.ts
```



```
|   └─ types.ts
|─ data/
|   └─ sample-volcano-chapter.txt
|   └─ demo-videos.json
|   └─ demo-lessons.json
|─ scripts/
|   └─ seed-videodb.ts
|   └─ verify-services.ts
|   └─ smoke-test.ts
|─ public/
|   └─ demo/
|─ instrumentation.ts
|─ casting.yaml
|─ casting.yaml.lock
|─ BUILD_STATUS.md
|─ DEMO_SCRIPT.md
|─ SUBMISSION_COPY.md
|─ README.md
|─ .env.example
```

8. Environment Variables

Create `.env.example`:

```
VIDEODB_API_KEY=
VIDEODB_COLLECTION_ID=

OPENAI_API_KEY=
OPENAI_MODEL=

SARVAM_API_KEY=
ELEVENLABS_API_KEY=
ELEVENLABS_VOICE_ID=

OTEL_SERVICE_NAME=kathaquest
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318
OTEL_EXPORTER_OTLP_TRACES_ENDPOINT=http://localhost:4318/v1/traces
OTEL_EXPORTER_OTLP_METRICS_ENDPOINT=http://localhost:4318/v1/metrics

SIGNOZ_URL=http://localhost:8080
SIGNOZ_MCP_URL=http://localhost:8000/mcp

DEMO_FORCE_ELEVENLABS_FAILURE=false
DEMO_MODE=true
```

Do not expose these variables to the browser unless explicitly safe.

9. Core Data Types

```
type LessonStatus =  
  | "queued"  
  | "extracting"  
  | "searching"  
  | "narrating"  
  | "compiling"  
  | "ready"  
  | "failed";  
  
type LearningConcept = {  
  id: string;  
  title: string;  
  explanation: string;  
  videoSearchQuery: string;  
  quiz: {  
    question: string;  
    options: string[];  
    correctAnswer: string;  
  };  
};  
  
type VideoEvidence = {  
  videoId: string;  
  videoTitle: string;  
  startSeconds: number;  
  endSeconds: number;  
  relevanceScore?: number;  
  sourceUrl?: string;  
  licence?: string;  
};  
  
type Episode = {  
  id: string;  
  conceptId: string;  
  title: string;  
  explanation: string;  
  streamUrl: string;  
  narrationUrl?: string;  
  narrationProvider?: "sarvam" | "elevenlabs";  
  evidence: VideoEvidence[];  
};  
  
type Lesson = {  
  id: string;  
  title: string;  
  ageGroup: string;
```

```
language: "en-IN" | "hi-IN";
status: LessonStatus;
concepts: LearningConcept[];
episodes: Episode[];
traceId?: string;
createdAt: string;
};
```

10. VideoDB Implementation

Seeding workflow

Create:

```
scripts/seed-videodb.ts
```

It should:

1. Read `data/demo-videos.json`.
2. Connect to VideoDB.
3. Upload each video.
4. Wait for processing.
5. Run spoken-word and visual understanding.
6. Index both outputs.
7. Save returned VideoDB IDs to a local cache file.
8. Avoid uploading the same video twice.

The Node.js SDK supports video upload from public URLs or local files, collection search, indexing and timeline assets.

Search workflow

Create a service:

```
searchEducationalArchive(query: string): Promise<VideoEvidence[]>
```

Requirements:

- Search across the VideoDB collection.
- Use visual and spoken-word information.
- Return a maximum of three relevant segments.
- Remove duplicate or overlapping moments.
- Prefer segments between 8 and 25 seconds.
- Record search latency.
- Record number of results.

- Record relevance scores when available.

Episode compilation

For every concept:

1. Select the best one to three segments.
2. Compile them into one playable stream.
3. Return the HLS stream URL.
4. Store source evidence in the lesson object.

Low-result fallback

When no useful result is found:

1. Ask the LLM to rewrite the search query.
2. Search again once.
3. Record `query_rewrite_used=true`.
4. If the second search fails, show a visible error instead of fake results.

11. LLM Structured Output

The LLM must return valid JSON.

Schema:

```
{
  "chapterTitle": "Volcanoes",
  "concepts": [
    {
      "title": "What is a volcano?",
      "explanation": "A volcano is an opening in the Earth...",
      "videoSearchQuery": "cross section of volcano magma chamber",
      "quiz": {
        "question": "Where is magma stored?",
        "options": [
          "Magma chamber",
          "Cloud",
          "Ocean",
          "Forest"
        ],
        "correctAnswer": "Magma chamber"
      }
    }
  ]
}
```

Rules:

- Exactly three concepts.
- Explanations must be suitable for the selected age.
- Avoid unsupported factual claims.
- Search queries must describe visual or spoken evidence.
- Do not generate long scripts.
- Validate all output with Zod.
- Retry once when validation fails.

12. Narration Provider Router

Implement:

```
generateNarration({
  text,
  language,
  forceFailure
}): Promise<{
  audioUrl: string;
  provider: "sarvam" | "elevenlabs";
  fallbackUsed: boolean;
}>
```

Routing rules

Hindi

1. Sarvam
2. ElevenLabs fallback if the configured model supports the text

English

1. ElevenLabs
2. Sarvam fallback

Observability requirements

For every provider request, record:

- Provider name
- Model name
- Language
- Input character count
- Request duration
- Success or failure
- HTTP status

- Retry count
- Fallback used
- Estimated cost where available

Never record API keys or complete child audio in telemetry.

13. OpenTelemetry and SigNoz Plan

Root trace

Create one root span:

```
lesson.generate
```

Child spans

```
document.extract  
llm.extract_concepts  
videodb.search_concept  
videodb.rewrite_query  
videodb.compile_episode  
sarvam.translate  
sarvam.speech_to_text  
tts.generate  
tts.fallback  
quiz.evaluate  
revision.compile  
lesson.persist
```

Required span attributes

```
lesson.id  
chapter.title  
chapter.character_count  
student.age_group  
lesson.language  
ai.provider  
ai.model  
ai.input_size  
ai.output_size  
video.query  
video.result_count  
video.relevance_score  
video.stream_generated  
tts.provider
```

```
tts.fallback_used  
pipeline.status  
error.type  
error.retryable
```

Do not store:

- Full book text
- Full child questions
- API keys
- Personal information
- Raw voice recordings

Custom metrics

Implement at least these metrics:

```
kathaquest.lesson.generated  
kathaquest.lesson.failed  
kathaquest.lesson.duration  
kathaquest.videodb.search.duration  
kathaquest.videodb.search.results  
kathaquest.videodb.empty_results  
kathaquest.tts.request.duration  
kathaquest.tts.failures  
kathaquest.tts.fallbacks  
kathaquest.questions.asks  
kathaquest.revision.generated
```

Recommended metric types:

- Counters for totals
- Histograms for durations
- Up/down counter only where necessary

Logs

Use structured JSON logs with:

- Timestamp
- Level
- Service name
- Trace ID
- Span ID
- Lesson ID
- Event
- Provider
- Error message

Avoid plain unstructured `console.log` for pipeline operations.

14. SigNoz Foundry Setup

Install SigNoz using Foundry as required by the hackathon.

```
curl -fsSL https://signoz.io/foundry.sh | bash
```

Create:

```
apiVersion: v1alpha1
kind: Installation
metadata:
  name: signoz
spec:
  deployment:
    flavor: compose
    mode: docker
  mcp:
    spec:
      enabled: true
```

Deploy:

```
foundryctl gauge -f casting.yaml
foundryctl cast -f casting.yaml
```

Verify:

```
docker ps
curl -fsS localhost:8000/livez
```

Expected services:

- SigNoz UI on port `8080`
- OTLP gRPC on port `4317`
- OTLP HTTP on port `4318`
- SigNoz MCP on port `8000`

Foundry generates `casting.yaml.lock`; both casting files must be committed for judging reproducibility. SigNoz currently documents Foundry as the supported self-host installation method.

15. SigNoz Dashboards

Create at least two dashboards.

Dashboard 1: Lesson Agent Health

Panels:

- Total lessons generated
- Lesson success rate
- P50 lesson duration
- P95 lesson duration
- Failed lessons
- Average VideoDB search latency
- Empty VideoDB searches
- TTS fallback count
- Recent error traces

Dashboard 2: AI Provider Reliability

Panels:

- Requests by provider
- Average latency by provider
- Failures by provider
- Fallbacks over time
- Requests by language
- Slowest pipeline stage
- Video results per search
- Revision reels generated

Optional dashboard

Learning Content Quality

Panels:

- Low-result searches
- Query rewrites
- Average number of evidence clips
- Questions asked
- Wrong answers by concept
- Revision generation success

Take screenshots of the dashboards for the README.

16. Alerts

Create at least three alerts.

Alert 1: Lesson generation failure

Trigger when:

```
kathaquest.lesson.failed > 0
```

Alert 2: TTS provider failure

Trigger when:

```
kathaquest.tts.failures > 0
```

Alert 3: No VideoDB results

Trigger when:

```
kathaquest.videodb.empty_results > 0
```

Optional alert:

- P95 lesson generation duration exceeds the demo threshold.

For the demo, use thresholds that can be triggered reliably.

17. User Interface

Visual direction

Build a playful but polished children's education interface.

Use:

- Large rounded cards
- Friendly typography
- Bright but accessible colours
- Clear progress indicators
- Simple language
- Minimal navigation
- Subtle animations

- Strong empty, loading and error states

Do not make it look like an enterprise dashboard.

Screen 1: Home

Include:

- Product name
- One-line explanation
- Upload area
- “Use demo volcano chapter” button
- Age selector
- Language selector
- “Create my video adventure” button

Screen 2: Generation progress

Display pipeline steps:

```
Reading your chapter
Finding key ideas
Searching real educational videos
Creating your episodes
Preparing narration
Finishing your lesson
```

Show completed, active and pending states.

Screen 3: Lesson

Include:

- Chapter title
- Three episode cards
- HLS player
- Listen button
- Evidence and source information
- Voice-question input
- Quiz
- Revision reel

Screen 4: Developer observability panel

This may be a drawer or separate page.

Display:

- Trace ID

- Total generation time
- VideoDB result count
- Voice provider
- Fallback status
- Service status
- Button to simulate failure
- Button to open SigNoz

This panel is important during the judging demo.

18. API Routes

POST /api/lessons/generate

Input:

```
{
  "chapterText": "...",
  "ageGroup": "8-10",
  "language": "hi-IN"
}
```

Output:

```
{
  "lessonId": "...",
  "status": "ready",
  "traceId": "..."
}
```

POST /api/questions/ask

Accept:

- Lesson ID
- Text question
- Or uploaded audio

Return:

- Transcript
- Simple answer
- Supporting VideoDB stream
- Evidence metadata

POST /api/quiz/submit

Accept answers and return:

- Score
- Incorrect concept IDs
- Revision reel URL

POST /api/demo/failure

Toggle a controlled provider failure for the next request.

Protect it behind:

DEMO_MODE=true

GET /api/health

Return the health of:

- Application
- VideoDB
- Sarvam
- ElevenLabs
- OpenTelemetry exporter

Never expose secrets.

19. One-Day Execution Roadmap

Phase 0: 20 minutes — Validate the rules and credentials

Tasks:

- Confirm VideoDB API key works.
- Confirm Sarvam API key works.
- Confirm ElevenLabs API key works.
- Confirm Docker is running.
- Create `.env.local`.
- Create `.env.example`.
- Initialise `BUILD_STATUS.md`.
- Create a public GitHub repository.
- Commit the starting state.

Stop immediately and fix credentials before building UI.

Phase 1: 90 minutes — Prove VideoDB vertical slice

Tasks:

- Install VideoDB SDK.
- Upload one real volcano video.
- Create transcript and visual understanding.
- Index it.
- Search for one concept.
- Generate a playable HLS stream.
- Display it in a minimal page.

Exit condition:

One real search query produces a playable exact video moment.

Do not continue until this works.

Phase 2: 75 minutes — Seed archive

Tasks:

- Prepare 5–8 videos.
- Add licence metadata.
- Build the seeding script.
- Upload and index the archive.
- Cache IDs.
- Search across the collection.
- Remove duplicate results.

Exit condition:

Each of the three volcano concepts returns at least one useful clip.

Phase 3: 60 minutes — Chapter understanding

Tasks:

- Add sample chapter.
- Add PDF text extraction.
- Add structured LLM output.
- Add Zod validation.
- Generate exactly three concepts and quizzes.

Exit condition:

Chapter input reliably returns valid structured concepts.

Phase 4: 75 minutes — Lesson generation

Tasks:

- Connect concepts to VideoDB search.
- Compile one stream per concept.
- Persist lesson locally.
- Return traceable progress states.

Exit condition:

Clicking one button creates three playable episode cards.

This is the minimum viable VideoDB submission.

Phase 5: 60 minutes — Sarvam and ElevenLabs

Tasks:

- Implement Sarvam TTS.
- Implement ElevenLabs TTS.
- Implement provider routing.
- Add a “Listen” button.
- Add controlled ElevenLabs failure.
- Add Sarvam fallback.

Exit condition:

English or Hindi narration plays, and forced fallback works.

Phase 6: 60 minutes — Voice questions

Tasks:

- Record browser audio.
- Send audio to Sarvam STT.
- Search VideoDB using the transcript.
- Return a supporting clip.
- Add typed-question fallback.

Exit condition:

A Hindi voice question returns a video answer.

If microphone handling takes too long, keep typed questions and provide one pre-recorded Hindi audio example.

Phase 7: 90 minutes — OpenTelemetry and SigNoz

Tasks:

- Install SigNoz through Foundry.
- Enable MCP.
- Add Node.js OpenTelemetry instrumentation.
- Export traces and metrics.
- Add manual pipeline spans.
- Add structured logs.
- Confirm one complete trace appears.

Exit condition:

One lesson request is visible end-to-end in SigNoz.

Phase 8: 60 minutes — Dashboards and alerts

Tasks:

- Create two dashboards.
- Create three alerts.
- Trigger the TTS failure.
- Verify the failed span and fallback.
- Capture screenshots.

Exit condition:

Judges can visually understand the system without reading code.

Phase 9: 45 minutes — Quiz and revision reel

Tasks:

- Render generated quizzes.
- Track incorrect answers.
- Search clips for wrong concepts.
- Compile one revision stream.

This is removable if the earlier core workflow is unstable.

Phase 10: 75 minutes — UI polish

Tasks:

- Improve typography and spacing.
- Add loading states.
- Add error recovery.
- Add mobile responsiveness.
- Add source evidence.

- Remove placeholder content.
- Add product branding.
- Test the complete demo path.

Avoid redesigning functioning components.

Phase 11: 60 minutes — Testing

Run these tests:

1. Demo chapter generates three concepts.
2. All episodes play.
3. Video sources are real and documented.
4. Hindi narration works.
5. Voice or typed question works.
6. TTS fallback works.
7. Trace reaches SigNoz.
8. Metrics appear.
9. Alerts can trigger.
10. No API key appears in browser responses.
11. Production build succeeds.
12. Fresh installation instructions work.

Run:

```
npm run lint
npm run typecheck
npm run build
npm run smoke-test
```

Phase 12: 90 minutes — Submission materials

Complete:

- README
- Demo recording
- Screenshots
- Architecture diagram
- VideoDB description
- SigNoz description
- AI usage disclosure
- Submission forms
- Final public repository
- Tagged release

Submit SigNoz first because its exact closing time is unclear.

Then submit VideoDB before 10:00 AM IST on July 27, 2026.

20. Scope-Cut Order

When behind schedule, remove features in this order:

1. Embedded narration overlay
2. Revision reel
3. Microphone recording
4. Automatic translation
5. Third dashboard
6. SigNoz MCP interaction
7. PDF upload

Do not remove:

- Real archived media
 - VideoDB indexing and search
 - Playable VideoDB output
 - Public repository
 - OpenTelemetry trace
 - SigNoz dashboard
 - Controlled failure and fallback
 - Required casting files
 - Submission documentation
-

21. Demo Script

Keep the final demonstration under four minutes.

Scene 1: Problem

Say:

Children often struggle with static textbook chapters, even though the internet already contains excellent educational footage. The problem is finding the exact moments that explain the chapter safely and quickly.

Scene 2: Generate lesson

- Open KathaQuest.
- Select the volcano chapter.
- Select age 9.
- Select Hindi.
- Click "Create my video adventure."

Scene 3: Show VideoDB intelligence

- Show three generated concepts.

- Play one compiled episode.
- Show its real source video.
- Show the timestamped evidence.
- Explain that VideoDB searched speech and scenes rather than returning a generic video link.

Scene 4: Ask a question

Ask:

“Lava bahar kyun aata hai?”

Show:

- Sarvam transcription
- Simple Hindi response
- Exact supporting video moment

Scene 5: Show observability

Open SigNoz.

Show:

- Root lesson trace
- VideoDB search spans
- LLM span
- TTS span
- Total latency
- Custom attributes
- Dashboard metrics

Scene 6: Simulate failure

- Activate ElevenLabs failure.
- Generate narration.
- Show failed span.
- Show fallback to Sarvam.
- Show successful final lesson.
- Show failure metric or alert.

Scene 7: Closing pitch

Say:

KathaQuest makes books visual, multilingual and evidence-based. VideoDB unlocks the knowledge inside real educational footage, while SigNoz makes the complete AI system observable, reliable and debuggable.

22. README Structure

Create a strong README with:

1. Product name and tagline
2. Demo screenshot or GIF
3. Problem
4. Solution
5. Main features
6. How VideoDB is used
7. How SigNoz is used
8. Architecture diagram
9. Technology stack
10. Local installation
11. Environment variables
12. VideoDB archive seeding
13. SigNoz Foundry installation
14. Dashboard screenshots
15. Failure recovery demonstration
16. Demo instructions
17. Public media sources and licences
18. AI assistant disclosure
19. Known limitations
20. Future roadmap
21. Team details

Include a visible section:

AI Tools Disclosure

Example:

We used OpenAI Codex as a coding assistant for implementation, debugging and documentation. All product decisions, integration choices, testing and final submission review were performed by the team.

23. VideoDB Submission Copy

Create a maximum 200-word version in `SUBMISSION_COPY.md`.

Suggested draft:

KathaQuest

KathaQuest turns textbook chapters into short, multilingual video learning journeys for children using real educational footage.

A parent or teacher uploads a chapter and selects the child's age and language. KathaQuest extracts the chapter's key concepts and converts them into media-search queries. VideoDB ingests and indexes a trusted archive of real educational videos using spoken-word and visual understanding. It then retrieves the exact moments that explain each concept and compiles them into playable micro-lessons.

Children can ask questions using text or Hindi voice input. Instead of returning only a generated text answer, KathaQuest searches the archive again and responds with a supporting video moment. A short quiz can also identify weak concepts and create a personalised revision reel from relevant clips.

VideoDB powers archive ingestion, speech and scene indexing, semantic search, timestamp retrieval, clip compilation and HLS delivery. Sarvam AI provides Indian-language speech and narration, while ElevenLabs provides expressive voice generation. The complete AI workflow is instrumented through OpenTelemetry and observed in SigNoz.

24. SigNoz Submission Positioning

Use this description:

KathaQuest is an observable multi-provider learning agent. SigNoz traces the complete journey from chapter extraction to VideoDB retrieval, translation, narration and video compilation. Custom metrics track lesson success, search quality, latency, provider failures and fallbacks. A controlled ElevenLabs failure demonstrates how SigNoz surfaces the failed span while KathaQuest automatically recovers through Sarvam AI. Foundry makes the complete observability stack reproducible.

Emphasise:

- Agent observability
 - External-tool tracing
 - Provider reliability
 - Failure recovery
 - Search quality
 - User-facing trust
 - Reproducible infrastructure
-

25. Definition of Done

The product is complete only when all critical items are true.

Product

- [] Sample chapter generates exactly three concepts
- [] At least five real videos are indexed
- [] Each concept produces a playable VideoDB stream
- [] Source footage and timestamps are visible
- [] English or Hindi narration works

- ☐ Question search returns supporting video evidence
- ☐ UI is usable without technical explanation

VideoDB

- ☐ Real archived media is used
- ☐ Spoken-word indexing is demonstrated
- ☐ Visual or scene indexing is demonstrated
- ☐ Semantic search is demonstrated
- ☐ Exact timestamp retrieval is demonstrated
- ☐ Clip or timeline compilation is demonstrated
- ☐ HLS playback works
- ☐ Public repository exists
- ☐ Description is within 200 words

SigNoz

- ☐ SigNoz installed using Foundry
- ☐ `casting.yaml` committed
- ☐ `casting.yaml.lock` committed
- ☐ Root trace is visible
- ☐ Provider spans are visible
- ☐ Custom metrics are visible
- ☐ Structured logs are visible
- ☐ Two dashboards are created
- ☐ Three alerts are configured
- ☐ Controlled failure is traceable
- ☐ Automatic fallback is demonstrated
- ☐ AI assistant use is disclosed

Engineering

- ☐ No secrets committed
- ☐ Production build succeeds
- ☐ Core paths have error handling
- ☐ Repository contains installation instructions
- ☐ Demo can be reproduced
- ☐ No mocked media output is used
- ☐ Final demo recording is complete
- ☐ Both submissions are sent

26. Post-Hackathon Product Roadmap

Do not implement this now. Include it only in the README or pitch.

Phase 1: Teacher pilot

- Teacher-curated video archives

- More subjects
- Chapter-to-lesson editing
- Teacher approval before publishing
- Learning outcome tracking
- Safer content review

Phase 2: School product

- School and classroom accounts
- Curriculum mapping
- Teacher dashboards
- Assignments
- Student progress
- Publisher integrations
- Per-school licensing

Phase 3: Personalised learning platform

- Student knowledge profiles
- Adaptive lesson length
- Regional-language expansion
- Voice-based tutoring
- Personalised revision libraries
- Parent reports
- Accessibility modes

Business customers

- Parents
- Schools
- Coaching institutes
- Teachers
- Educational publishers
- Regional learning platforms

Business model

- Parent subscription
- Per-student school licence
- Teacher creator plan
- Publisher API and white-label licensing
- Enterprise archive transformation

Long-term differentiation

KathaQuest's defensibility should come from:

- Trusted educational media collections
- Curriculum-to-video retrieval quality
- Multilingual Indian-language experience

- Evidence-based answers
 - Learning effectiveness data
 - Observable and auditable AI workflows
-

Final Engineering Principle

Do not attempt to build the full business in one day.

The submission must tell one clear story:

A child uploads a chapter, VideoDB finds the exact moments from real educational footage that explain it, Sarvam and ElevenLabs make it accessible through voice, and SigNoz makes the complete AI pipeline reliable and transparent.

Build this journey extremely well.